Mateo Biggs
DS 4420
Final Project - Bayesian Model

2026-04-15

Libraries

```
library(jsonlite)
library(ggplot2)
library(dplyr)
```

Combine All Seasons (2008–2025), Divisions 1–3

```
years <- 2011:2025
years <- years[years != 2020]  # remove COVID-shortened season

divisions <- c("div1", "div2", "div3")

all_rows <- list()
row_i <- 1

for (div in divisions) {
  for (yr in years) {
    path <- paste0("data/", div, "/", yr, ".json")
    if (!file.exists(path)) next

    raw <- fromJSON(path)

    for (team_key in names(raw)) {
      team_info <- as.data.frame(raw[[team_key]], stringsAsFactors = FALSE)
      team_info$team_key <- team_key
      team_info$season <- yr
      team_info$division <- div
      all_rows[[row_i]] <- team_info
      row_i <- row_i + 1
    }
  }
}

all_data <- bind_rows(all_rows)
```

Feature Engineering

```
model_data <- data.frame(
  team = all_data$team,
  conference = all_data$league,
  division = all_data$division,
  season = all_data$season,
  wpct = as.numeric(all_data$WPCT),
  r_batting = as.numeric(all_data$R..Batting.),
  r_pitching = as.numeric(all_data$R..Pitching.),
  g = as.numeric(all_data$G),
  stringsAsFactors = FALSE
)

model_data$rbat_z <- (model_data$r_batting - mean(model_data$r_batting, na.rm = TRUE)) / sd(model_data$
model_data$rpit_z <- (model_data$r_pitching - mean(model_data$r_pitching, na.rm = TRUE)) / sd(model_data

model_data <- model_data[complete.cases(model_data$wpct, model_data$rbat_z, model_data$rpit_z), ]
```

Bayesian Model Setup

Season-varying coefficients for R (Batting) and R (Pitching), conference random effects, and a shared intercept — all estimated via Metropolis-Hastings.

```
unique_seasons <- sort(unique(model_data$season))
unique_confs <- sort(unique(model_data$conference))
n_seasons <- length(unique_seasons)
n_confs <- length(unique_confs)

# intercept + batting effect per season + pitching effect per season + conference random effects + sigm
n_params <- 1 + n_seasons + n_seasons + n_confs + 1
```

Log-Likelihood

```
log_likelihood <- function(params, data) {
  intercept <- params[1]
  beta_bat <- params[2:(1 + n_seasons)]
  beta_pit <- params[(2 + n_seasons):(1 + 2 * n_seasons)]
  conf_effects <- params[(2 + 2 * n_seasons):(1 + 2 * n_seasons + n_confs)]
  sigma <- exp(params[length(params)])

  ll <- 0
  for (i in 1:nrow(data)) {
    s_idx <- which(unique_seasons == data$season[i])
    c_idx <- which(unique_confs == data$conference[i])

    mu <- intercept + beta_bat[s_idx] * data$rbat_z[i] + beta_pit[s_idx] * data$rpit_z[i] + conf_effects
    ll <- ll + dnorm(data$wpct[i], mean = mu, sd = sigma, log = TRUE)
  }
```

```
  return(ll)
}
```

Log-Prior

```
log_prior <- function(params) {
  intercept <- params[1]
  beta_bat <- params[2:(1 + n_seasons)]
  beta_pit <- params[(2 + n_seasons):(1 + 2 * n_seasons)]
  conf_effects <- params[(2 + 2 * n_seasons):(1 + 2 * n_seasons + n_confs)]
  log_sigma <- params[length(params)]

  lp <- dnorm(intercept, mean = 0.5, sd = 0.2, log = TRUE)
  lp <- lp + sum(dnorm(beta_bat, mean = 0, sd = 0.5, log = TRUE))
  lp <- lp + sum(dnorm(beta_pit, mean = 0, sd = 0.5, log = TRUE))
  lp <- lp + sum(dnorm(conf_effects, mean = 0, sd = 0.1, log = TRUE))
  lp <- lp + dnorm(log_sigma, mean = log(0.1), sd = 1, log = TRUE)

  return(lp)
}
```

Log-Posterior

```
log_posterior <- function(params, data) {
  return(log_likelihood(params, data) + log_prior(params))
}
```

Metropolis-Hastings Sampler

```
run_mcmc <- function(data, n_iter = 5000, burn_in = 1000) {

  current <- rep(0, n_params)
  current[1] <- 0.5
  current[length(current)] <- log(0.1)

  proposal_sd <- rep(0.0005, n_params)
  proposal_sd[1] <- 0.003
  proposal_sd[length(current)] <- 0.003

  samples <- matrix(NA, nrow = n_iter, ncol = n_params)
  current_lp <- log_posterior(current, data)
  accepted <- 0

  cat("Running MCMC...\n")
  for (iter in 1:n_iter) {

    proposal <- rnorm(n_params, mean = current, sd = proposal_sd)
```

```r
    proposed_lp <- log_posterior(proposal, data)

    log_ratio <- proposed_lp - current_lp
    if (log(runif(1)) < log_ratio) {
      current <- proposal
      current_lp <- proposed_lp
      accepted <- accepted + 1
    }

    samples[iter, ] <- current

    if (iter %% 1000 == 0) {
      cat("Iteration", iter, "of", n_iter, "- acceptance rate:", round(accepted / iter, 3), "\n")
    }
  }

  cat("Done! Final acceptance rate:", round(accepted / n_iter, 3), "\n")

  samples <- samples[(burn_in + 1):n_iter, ]
  return(samples)
}
```

Run the Sampler

```r
set.seed(42)
samples <- run_mcmc(model_data, n_iter = 10000, burn_in = 5000)
```

```
## Running MCMC...
## Iteration 1000 of 10000 - acceptance rate: 0.449
## Iteration 2000 of 10000 - acceptance rate: 0.45
## Iteration 3000 of 10000 - acceptance rate: 0.427
## Iteration 4000 of 10000 - acceptance rate: 0.396
## Iteration 5000 of 10000 - acceptance rate: 0.36
## Iteration 6000 of 10000 - acceptance rate: 0.323
## Iteration 7000 of 10000 - acceptance rate: 0.298
## Iteration 8000 of 10000 - acceptance rate: 0.274
## Iteration 9000 of 10000 - acceptance rate: 0.257
## Iteration 10000 of 10000 - acceptance rate: 0.245
## Done! Final acceptance rate: 0.245
```

Posterior Summary

```r
intercept_draws <- samples[, 1]
cat("Intercept mean:", mean(intercept_draws), "\n")
```

```
## Intercept mean: 0.4892155
```

```r
cat("Intercept 95% CI:", quantile(intercept_draws, c(0.025, 0.975)), "\n")
```

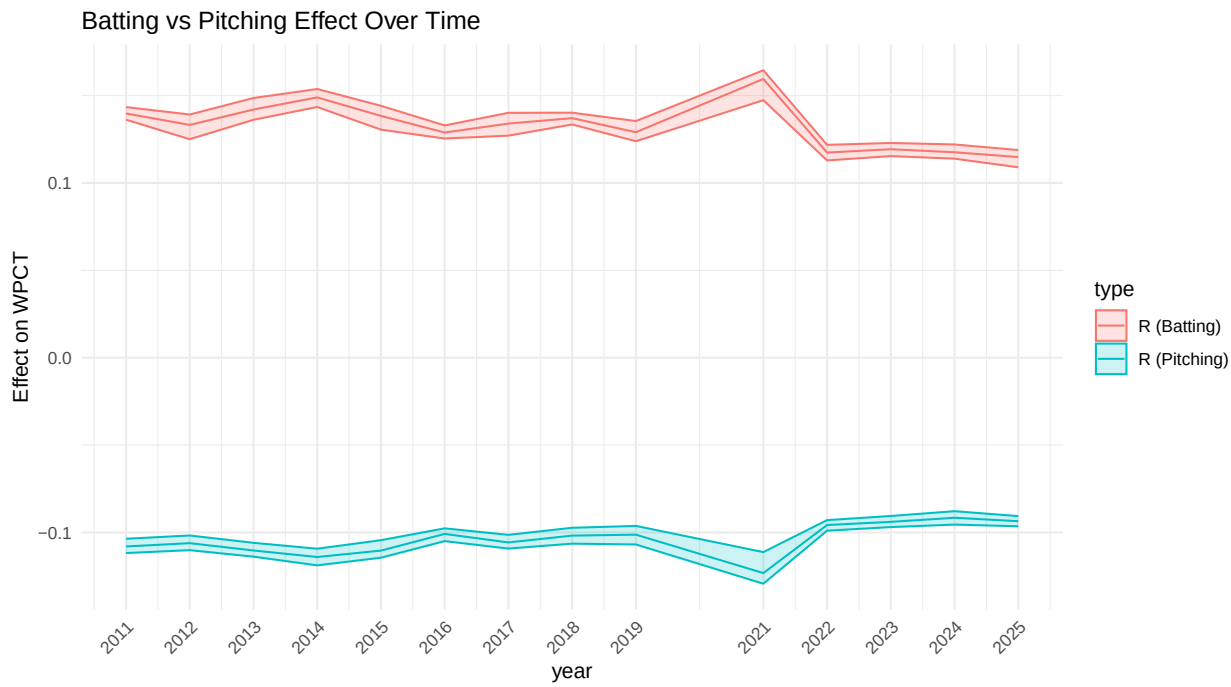
```
## Intercept 95% CI: 0.4876263 0.4911928
```

Extract Season-Specific Effects

```r
bat_means <- c()
bat_lo <- c()
bat_hi <- c()
pit_means <- c()
pit_lo <- c()
pit_hi <- c()

for (s in 1:n_seasons) {
  bat_col <- 1 + s
  pit_col <- 1 + n_seasons + s

  bat_means[s] <- mean(samples[, bat_col])
  bat_lo[s] <- quantile(samples[, bat_col], 0.025)
  bat_hi[s] <- quantile(samples[, bat_col], 0.975)

  pit_means[s] <- mean(samples[, pit_col])
  pit_lo[s] <- quantile(samples[, pit_col], 0.025)
  pit_hi[s] <- quantile(samples[, pit_col], 0.975)
}
```

Batting vs. Pitching Effect Over Time

```r
plot_df <- data.frame(
  year = rep(unique_seasons, 2),
  effect = c(bat_means, pit_means),
  lo = c(bat_lo, pit_lo),
  hi = c(bat_hi, pit_hi),
  type = rep(c("R (Batting)", "R (Pitching)"), each = n_seasons)
)

ggplot(plot_df, aes(x = year, y = effect, color = type)) +
  geom_line() +
  geom_ribbon(aes(ymin = lo, ymax = hi, fill = type), alpha = 0.2) +
  labs(y = "Effect on WPCT", title = "Batting vs Pitching Effect Over Time") +
  theme_minimal() +
  scale_x_continuous(breaks = unique_seasons) +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```




```r
cat("Total team-season observations:", nrow(model_data), "\n")
```

```
## Total team-season observations: 12261
```

```r
cat("Unique conferences:", length(unique(model_data$conference)), "\n")
```

```
## Unique conferences: 130
```

```r
cat("Unique teams:", length(unique(model_data$team)), "\n")
```

```
## Unique teams: 1000
```

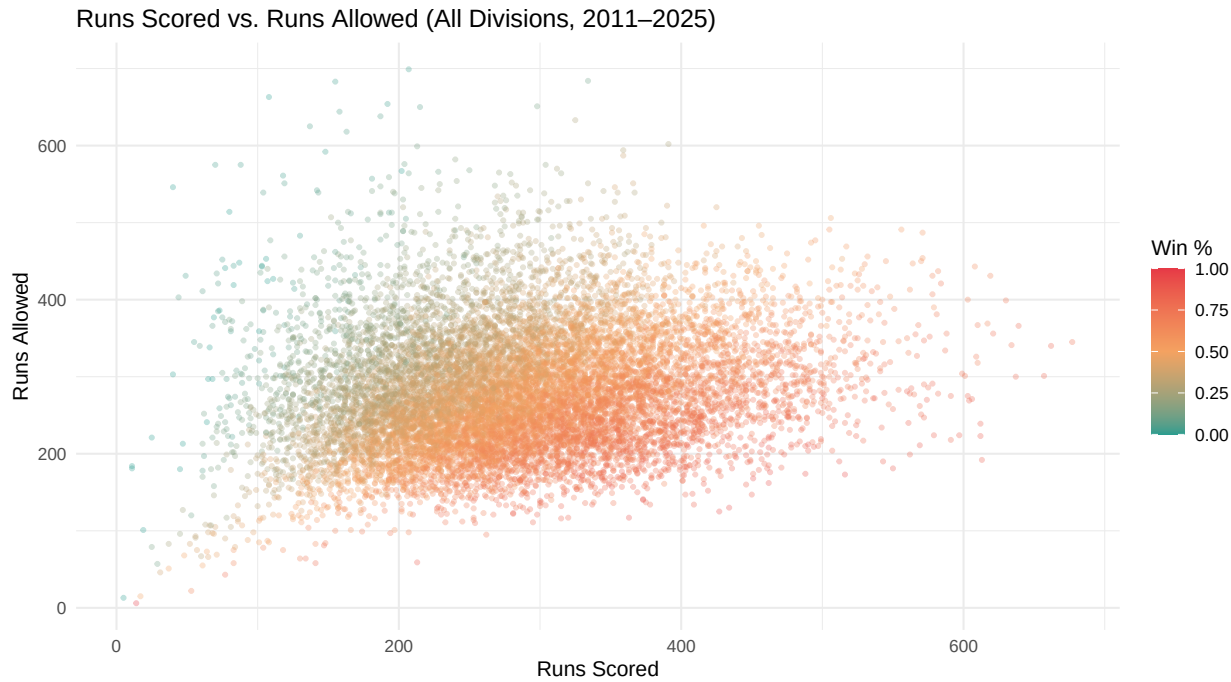
```r
cat("Seasons:", paste(range(model_data$season), collapse = "-"), "\n")
```

```
## Seasons: 2011-2025
```

```r
cat("Observations per division:\n")
```

```
## Observations per division:
```

```r
print(table(model_data$division))
```

```
##
## div1 div2 div3
## 3540 3470 5251
```

```r
ggplot(model_data, aes(x = r_batting, y = r_pitching, color = wpct)) +
  geom_point(alpha = 0.3, size = 0.8) +
  scale_color_gradient2(low = "#2A9D8F", mid = "#F4A261", high = "#E63946", midpoint = 0.5) +
  labs(
    x = "Runs Scored",
    y = "Runs Allowed",
    title = "Runs Scored vs. Runs Allowed (All Divisions, 2011-2025)",
    color = "Win %"
  ) +
  theme_minimal()
```